Classes

`Classes` are similar to `structs` in a lot of ways. A good way to think about what a `class` or `struct` is, would be to picture it as a blueprint. You create an outline of what you need, and then change the values tied to that outline in order to fit a certain situation. It is a way to visualize, manage, and manipulate data.

We briefly mentioned Object-Oriented Programming in our `structs` lesson. `Classes` are what use Object-Oriented Programming in swift. So though swift is a POP language, it supports OOP as well.

Both classes and structs can:

1. Define properties to store values.
2. Define methods to provide functionality
3. Define subscripts to provide access to their values using subscript syntax.
4. Define `initializers` to set up their initial state
5. Be extended to expand their functionality beyond their default implementation.
6. Conform to protocols to provide standard functionality of a certain kind.

However, though they have a lot of similarities, they have differences as well. Classes specifically:

1. You can make classes build upon functionality of another class, gaining all it's properties and methods to start. You can also override existing functionality, this is called Class Inheritance.
2. You don't get a default (member-wise) `initializer` in classes. So you always need to create your own `init`.
3. Classes differ from all other types we have covered in that they are reference types. Or copy by reference. This means that when we try to make a copy, instead we make a new reference to the same value. Meaning, if we change the “copy” it modifies the original as well.
4. Classes have what is called a `deinitializer`, which will run a block of code when the class is no longer in memory.
5. Even if you store a class in a constant, you can change any variable properties on it.

Structs and classes have very similar syntax. You create a class using the `class` keyword instead of struct. When we are naming a class we use `UpperCamelCase` as we are making a new type.

To reiterate, classes have all the same things as a struct.

1. Properties

2. Methods
3. Property Observers
4. Inits
5. Static Properties
6. Lazy Properties
7. Access Control

The only thing they don't have is mutating functions. This is because of how they work, they actually don't need them. You can modify the values store in a class in any function!

This lecture will focus primarily on the differences between structs and classes. So let's go ahead and write our first class.

```
class Dog {
    // Dog is a very basic idea, so what things might all dogs have in common?
    var name: String
    var breed: String

    // Since we are making a class we need to write an init.
    init(named name: String, breed: String) {
        self.name = name
        self.breed = breed
    }

    // And all dogs can bark right?
    func bark() {
        // Lets say that by default all dogs go woof.
        print("Woof")
    }
}
```

Now that we have made a `Dog` class, let's go ahead and make an instance of it. We do that in the same way we would with a struct. The types name followed by its `initializer`.

```
let rocco = Dog(named: "Rocco", breed: "Mutt")

// Since we made a dog, we can make it bark right?
rocco.bark()
```

Alright, we have made a new `Dog` class; however, people typically refer to their dogs based on its breed. They are all still dogs right, but they each have their own

characteristics based on their breed.

```
class YellowLab: Dog {

}
```

Yellow labs are pretty basic dogs, they all probably go woof, and they don't have any new special characteristics about them. But now if we try to make a yellow lab the `init` is going to ask us for a dog breed, but we know the dog breed right? It is a yellow lab, so let's fix this. By adding an `init` we can define the breed in the body of the `init` as opposed to keeping the same `init` from `Dog`.

```
class YellowLab: Dog {
    init(named name: String) {
        super.init(named: name, breed: "Yellow Lab")
    }
}
```

When we are writing an `init` for a child class, there is an important piece of syntax we need to use. Because we are a child class, that means that we need to fill out all the properties given to us from our parent class. We don't always have direct access to this information, so we cannot do it manually. Instead, we have to use our parent's `init` as part of ours. Whenever we need to reference what our parent is doing we use something similar to the `self` keyword, this new keyword is `super`. `super` is just a way of saying my parent. So within our `init` we have to call our parent's `init` by saying `super.init` and then passing any parameters it requires.

```
// Before sub class
let gracie = Dog(named: "Gracie", breed: "Yellow Lab")

// After
let fido = YellowLab(named: "Fido")
```

Even though we didn't explicitly declare a bark function or any properties inside our `YellowLab` class, we can still access all the ones that `Dog` provides. This is because `YellowLab` is a child / subclass of `Dog`.

```
fido.bark()
print(fido.name)
print(fido.breed)
```

Let's do another example of a breed of dog. A Shiba Inu.

```
final class ShibaInu: Dog {
    init(named name: String) {
        super.init(named: name, breed: "Shiba Inu")
    }
}
```

Here we have a new keyword, `final`. What `final` does is tells swift that there can't be any sub-classes of this class. So where `Dog` isn't marked as `final`, we can make sub-classes of it; however, since `ShibaInu` is a `final` class, we cannot subclass it.

Now, Shibas don't really “woof”, they more so scream. So they may “bark”, but a little different from our typical dog. How can we fix that? Well, with classes and class inheritance, we can override our bark method to do something different. In this case, instead of printing “woof” we want it to print out “screech”. We can use the override keyword to specify that we want to override a piece of functionality given to us by our parent.

```
// Inside our existing class...
final class ShibaInu: Dog {
    // Other Code...
    override func bark() {
        print("Screech")
    }
}
```

Above, we completely changed the functionality to our bark function. Sometimes, however, we may just want to add more functionality without changing the original. We can do that by using the `super` keyword again. We can call the original function given by our parent using `super.bark()` or `super` followed by the name and parameters of the function. So if we added `super.bark()` into our overridden `bark` function it would print both “screech” and “woof”.

```
let sinnoh = ShibaInu(named: "Sinnoh")
sinnoh.bark()
```

Since the `bark` function was overridden by the `ShibaInu` class, when we call `sinnoh.bark()` it uses the overridden version henceforth printing “Screech”.

```
// We can inherit from YellowLab
class Mutt: YellowLab {

}
```

```
// We cannot inherit from ShibaInu as it is a final class.
// ERROR: Inheritance from a final class 'ShibaInu'
class JapaneseMutt: ShibaInu {

}
```

Start Assignment Section 1

Treating Classes as Their Parent Type

So we have 4 different instances of Dogs right? We have Sinnoh, Fido, Gracie, and Rocco. I want to make them all bark. How might I do that? They are all dogs, yes they are different breeds, but they are all sub-classes of Dog. This means that we can actually treat them as if they were all just `Dogs`, so anywhere that we use the `Dog` type, we could use one of these subclass types there as well. This means we could store all of our dogs inside of an `Array`.

```
print("Who let the dogs out?")
let dogs = [sinnoh, fido, gracie, rocco]

for dog in dogs {
    dog.bark()
}

// "Screech"
// "Woof"
// "Woof"
// "Woof"
```

Classes Are Reference Types

Unlike all the types that we have covered so far. Classes are reference types, this means that when we copy them we copy them by reference instead of by their value. This means that whenever we try to make a copy instead of making a brand-new instance of the class, we are simply adding a new way to refer to the original. This is better shown than told in my opinion, so let's do some examples.

```
class Thermostat {
    var temperature = 0

    init(temperature: Int = 0) {
        self.temperature = temperature
```

```
    }
}

let house = Thermostat(temperature: 72)
let oven = house
oven.temperature = 400

print("House: \(house.temperature)")
print("Oven: \(oven.temperature)")
```

Unlike when we did this example for the structs lecture, our output is going to be:

```
"House: 400"
"Oven: 400"
```

With structs, as they are value types, we are making a brand-new thermostat that just happens to use the same information. With reference types, we are saying that we can refer to the same information differently. So in the above example we only have one thermostat, so when we modify the oven it modifies the house.

You will also notice that I made `oven` a constant; however, I was still able to modify its `temperature` property. Since classes are reference types, when we modify their properties we don't have to make a new instance, so they are always mutable as long as their properties are. I still won't be able to set the constants to a whole new instance of the class though.

Let's do the other example we did for structs.

```
class Person {
    var name: String
    var age: Int

    init(name: String, age: Int) {
        self.name = name
        self.age = age
    }

    func birthdayParty() {
        self.age += 1
    }
}

let personA = Person(name: "Joe", age: 24)
```

```
let personB = personA

print(personA.age)
print(personB.age)

personB.birthdayParty()
personA.name = "Juan"

// Classes are reference types, so what is going to be printed here?
print(personA.name, personA.age)
print(personB.name, personB.age)
```

Again, when we make a copy of a reference type, we are simply adding a new way to refer to the same data. So when we have `personB` have a birthday party, and have `personA` change his name, it is working with the exact same data, so our output would be:

```
"Juan", 25
"Juan", 25
```

Deinitializers

Swift classes can choose to have what is called a `Deinitializer` or a `deinit` for short. A `deinit` is effectively the opposite of an `init`. Where an `init` is called to create an instance of an object, a `deinit` is called when that instance gets destroyed. `Deinits` can never take parameters, or return data. They also aren't written with parens. The `deinit` of a class will automatically be called when the final copy of a class is out of scope.

For example, if you create an instance inside of a class inside a function, the instance only exists for the life of the function. When the function ends, the variable that stores the class goes out of scope, when this happens the `deinit` is called.

We never call `deinits` directly, they will automatically be called by the system. Structs cannot have `deinits`, as they won't ever have multiple references to them at a time, because they are copy by value.

Let's do an example showing how `deinits` may be called.

```
class MovieActor {
    static var moviesMade = 0

    let name: String

    init(named name: String) {
```

```
        self.name = name
        print("\(name) is alive!")
    }

    func makeMovie() {
        MovieActor.moviesMade += 1
        print("Hi, I'm \(name), and I'm making movie #\
(MovieActor.moviesMade)")
    }

    deinit {
        print("\(name) is dead =(")
    }
}
```

Now, let's use our new `MovieActor` class in a way that will show off both its `init` and `deinit`.

```
for _ in 1...3 {
    // We make our actor here, the init is called.
    let actor = MovieActor(named: "Ryan Reynolds")

    // We call makeMovie
    actor.makeMovie()

    // We reach the end of our iteration, so our actor constant
    // will be leaving scope.
    // Deinit is called.
}
```

Run the code, and see what the output looks like!

Overview

1. Classes and Structs are similar, they both let you create your own types with properties and methods.
2. A class can inherit from another class (parent, or super), and it gains all the properties and methods of its parent, including the parent's parent, and so on.
3. You can have long chains of parent / child class relationships. This is called a class hierarchy.
4. You can mark a class as final, this means you can no longer inherit from it.
5. You can override methods, this lets a child class replace or expand on the method of a parent class with a new implementation.

6. When you copy a class from one variable or constant to another, both of the variables or constants are pointing to the same memory, so changing one changes the other. This is because classes are copy by reference.
7. Classes have looser constant rules that structs. If a property is declared as a variable, it can be changed regardless of where the class instance is stored.